Milestone 1 Update

Please let us know if you'd like the report in a different format for next time!

Authors: Hector Astrom, Richard Lian 04/08/26

View our repository here:

https://github.com/r-lian/6.5930-Edge-Accelerator

Selecting our Accelerator

We looked into:

1. Coral NPU (RISC-V and old micro TPU version)
2. AMD AIE ML
3. Ethos U-55

We also briefly looked into the Jetson Nano, and STM32N6

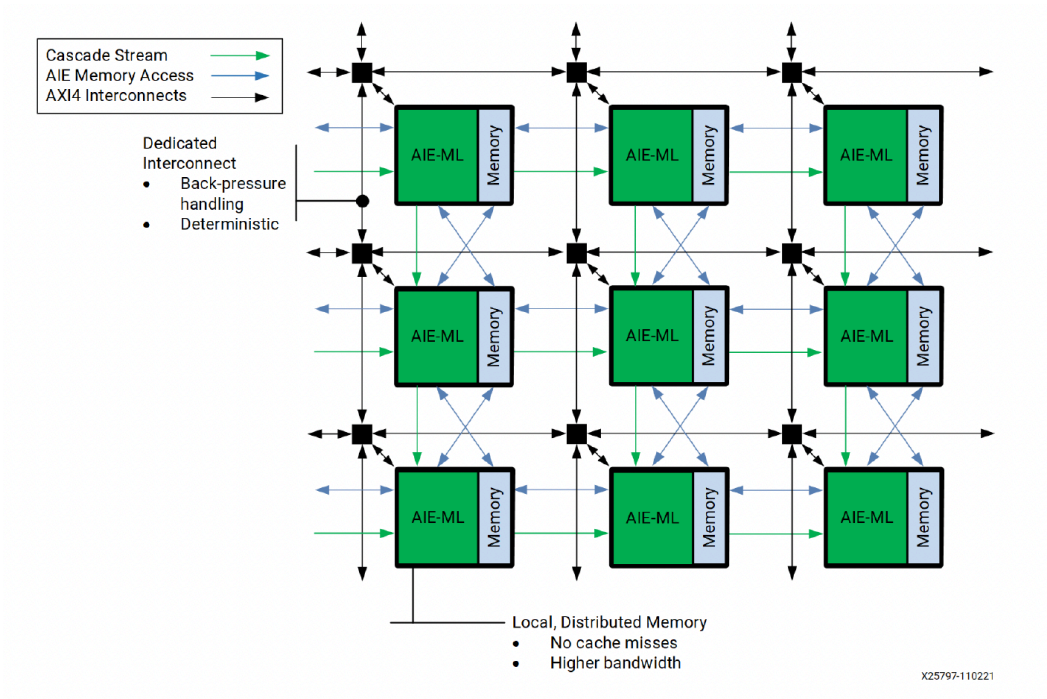
There were drawbacks with modeling all of these in AccelForge. When selecting our accelerator, we were optimizing for the following priorities:

1. Available public, explicit information about design and stats
2. Fit our size and energy requirements (ideally <1MB SRAM and <100mW, but also note the approved examples in the project description)
3. Can be modeled readily with the AccelForge components that we have (e.g. separate Memory and Compute blocks)

TLDR: The Ethos U-55 was the best fit. Although there were many things we still had to infer about this accelerator in terms of statistics (e.g. energy usage, area, bandwidth), the architecture was amenable to AccelForge modeling.

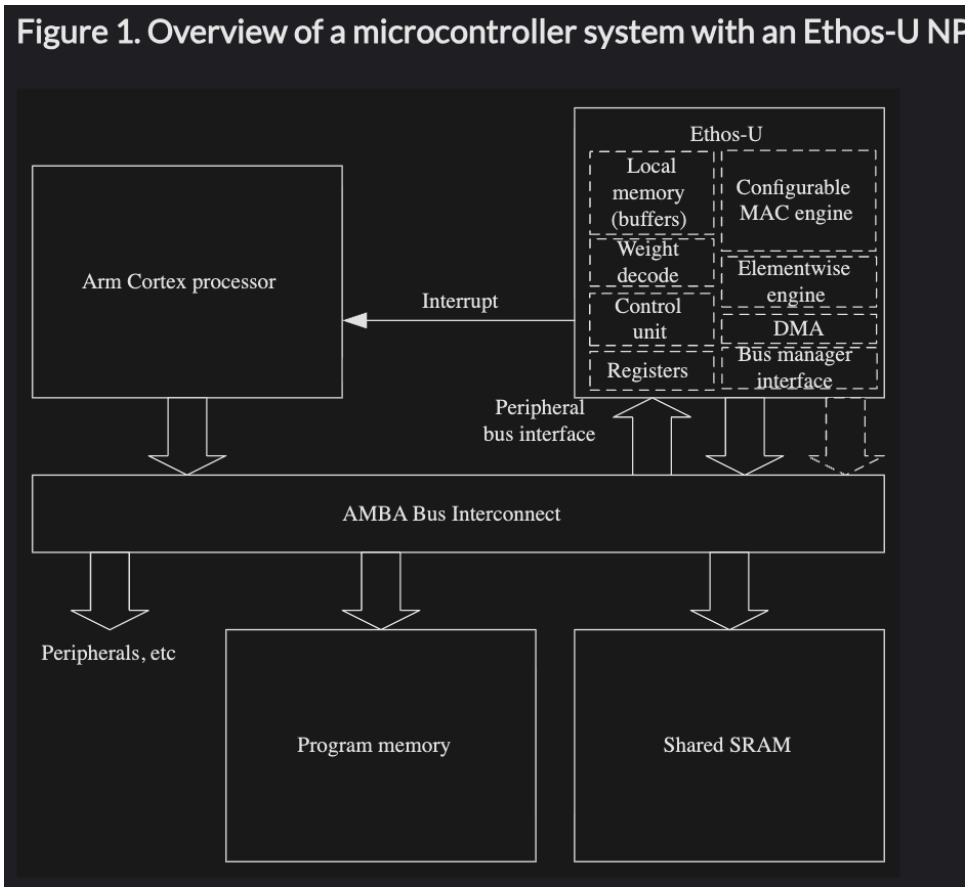
As for the other two accelerators:

1. The Coral NPU lacked public information. The TPU architecture is well documented, but the low-power edge Coral only has developer docs. They lack specifics beyond marketing claims
2. The AMD AIE ML was by far the best documented of the three accelerators. However, the architecture was chiefly tile based, and therefore hard to model with the AccelForge components we had



AIE ML tile based architecture, from "**Versal Adaptive SoC AIE-MLArchitecture Manual (AM020)**"

Ethos U-55 `Arch` Modeling



Main resources

1. Hardware architecture
2. Technical Reference Manual
3. Technical Overview Slidedeck

▼ Fact Sheet used for AccelForge modeling

1. Hard Silicon Constants (Quoted Public Facts)

- **Compute Configurations:** 32, 64, 128, or 256 MACs/cycle. [Source: Hot Chips 2020, Slide 11]
- **Performance Baseline:** 64 to 512 GOP/s at 1 GHz. [Source: Arm Product Page]
- **Peak Performance / Area Anchors:** ~0.5 TOP/s maximum performance; scales down to ~0.1 mm² in 16nm. [Source: Arm AI Blog]

- **Datapath Blocks:** MAC engine, weight decoder, DMA, shared buffer, central control, elementwise engine. [Source: Hot Chips 2020, Slide 11]
- **Memory Placement Paradigm:** Activations and scratchpad memory operate in System SRAM; compressed weights and command stream operate in System Flash. [Source: Hot Chips 2020, Slide 12]
- **Hardware Interfaces:** * 1× 32-bit APB slave (for registers).
    - 1x AXI M0 master (Read/Write access to SRAM).
    - 1x AXI M1 master (Read-Only access to Flash). [Source: Hot Chips 2020, Slide 13]
- **Quantization Format:** Limited to 8-bit or 16-bit integers natively. [Source: Arm AI Blog]
- **Operator Support & Fallback:** Native bit-accurate hardware support for 44 specific TensorFlow Lite operators. Unsupported operators automatically fall back to the host CPU (Cortex-M). [Source: Arm CV Blog]
- **Output Buffers & Control:** Internal OB0 and OB1 are fixed at 1 KB each. Controlled via a stream of 16-bit instructions.

2. Software & System Presets (Not Silicon Constants)

- **Arm Vela Compiler Memory Presets:** Deep Embedded (SRAM: 1.6 GB/s, Flash: 0.1 GB/s) and High-End Embedded (SRAM: 4 GB/s, Flash: 0.5 GB/s). *Note: These are compiler bounds, not physical silicon limits.* * **Standard Reference Board Specs:** Typical configurations pair the Cortex-M55 and Ethos-U55 at 400 MHz, backed by ~512 KB TCM, ~2 MB SRAM, and ~16 MB external Flash. [Source: Arm CV Blog]
- **Memory Bounds Warning:** Fully Connected layers are heavily bottlenecked by Flash bandwidth due to low weight reuse; scaling up to the 256-MAC config yields diminishing returns without matching bandwidth increases.

3. Model Assumptions (Unpublished by Arm)

These are all unpublished and are treated as assumptions.

- **Total Internal Shared Buffer Size:** (Dynamic depending on IFM/OFM needs, exact absolute limits per config vary by OEM license).

- **Block-Level Energy:** Specific per-MAC (pJ/op), per-AXI-read, and per-SRAM-read energy consumption.
- **Granular Area Breakdown:** The exact μm^2 area of individual internal components (DMA vs. Weight Decoder vs. MAC array).

Architecture in AccelForge

We keep the AccelForge architecture deliberately minimal. There are many versions of the Ethos with different MAC sizes or SRAM sizes, so we leave that as a modular piece with an accompanying Python script that populates the Jinja.

High level overview

Broadly, the architecture is organized into

- `FLASH`: slow upper memory
- `SRAM`: larger on-chip working memory
- `Scratchpad`: small fast local buffer
- `MAC`: configurable-width int8 MAC array

The scratchpad is meant to model the internal memory of the NPU, and has extremely high bandwidth. The MAC array captures the NPU's configurable compute width, while the memory hierarchy reflects the public Ethos-U55 deployment model in which activations are primarily staged through SRAM and weights are often sourced from slower nonvolatile storage.

Modularity

First, the **compute block is parameterized**. The MAC array width can be set to 32, 64, 128, or 256 MACs per cycle, so the same architecture template can represent multiple Ethos-U55-class configurations rather than a single fixed design.

Second, the **memory hierarchy is modular**. The model separates Flash, system SRAM, and a local scratchpad into distinct components, each with its own size,

bandwidth, area, and access energy. This allows individual memory levels to be resized or reparameterized without changing the rest of the architecture.

Third, the **system configuration is modular**. The loader exposes preset bandwidth modes and SRAM-capacity modes, so the same architecture can be evaluated under different deployment assumptions, such as more bandwidth-limited or more SRAM-rich systems.

Fourth, the **assumptions are modular**. Quantities that are not publicly specified, such as local-buffer size, per-access energy, and area density, are isolated as top-level parameters in the loader. As a result, the model can be recalibrated or sensitivity-tested without rewriting the YAML itself.

Assumptions

The model intentionally abstracts away **command-stream execution, weight compression/decompression**, and **CPU fallback** for unsupported operators. These mechanisms are treated as control overhead rather than explicit architectural blocks, since they are not naturally represented in AccelForge and are not required for first-order mapping analysis.

YAML

```
arch:
  # Ethos-U55 as an AccelForge model:
  #
  # - FLASH is a slow upper memory for weights / inputs / out
puts.
  # - SRAM is the larger on-chip buffer that the NPU and host
share or carve out.
  # - Scratchpad is a small, fast local working buffer near c
ompute.
  # - MAC is an int8 MAC array with configurable width.
  #
  # Assumptions that are not public Ethos-U55 constants are p
assed in through Jinja.
  nodes:
  - !Memory
```

```
    name: FLASH
    size: inf
    # Public Ethos-U55 systems use Flash as the slow upper me
mory
    # We keep write enabled here only so generic workloads wi
th final outputs remain legal
    total_latency: "max(read_actions * {{ BITS_PER_VALUE }} /
{{ FLASH_READ_BW_BITS_PER_S }}, write_actions * {{ BITS_PER_V
ALUE }} / {{ FLASH_WRITE_BW_BITS_PER_S }})"
    leak_power: 0
    area: 0
    actions:
    - {name: read, energy: {{ FLASH_READ_ENERGY_J }}, latenc
y: 0}
    - {name: write, energy: {{ FLASH_WRITE_ENERGY_J }}, laten
cy: 0}
    tensors: {keep: ~Intermediates, may_keep: All}

  - !Memory
    name: SRAM
    size: {{ SYSTEM_SRAM_SIZE_BITS }}
    # Public Vela presets expose SRAM bandwidths such as 1.6
GB/s and 4.0 GB/s.
    total_latency: "max(read_actions * {{ BITS_PER_VALUE }} /
{{ SRAM_READ_BW_BITS_PER_S }}, write_actions * {{ BITS_PER_VA
LUE }} / {{ SRAM_WRITE_BW_BITS_PER_S }})"
    leak_power: 0
    area: {{ SYSTEM_SRAM_AREA_M2 }}
    actions:
    - {name: read, energy: {{ SRAM_READ_ENERGY_J }}, latency:
0}
    - {name: write, energy: {{ SRAM_WRITE_ENERGY_J }}, latenc
y: 0}
    tensors: {keep: All, may_keep: All}

  - !Memory
```

```
    name: Scratchpad
    size: {{ LOCAL_BUFFER_SIZE_BITS }}
    # This stands in for the NPU's hidden local working stora
ge
    # We intentionally give it high bandwidth so it does not
become the bottleneck
    # unless you choose to make it small enough that refetchi
ng dominates.
    total_latency: "max(read_actions * {{ BITS_PER_VALUE }} /
{{ SCRATCH_READ_BW_BITS_PER_S }}, write_actions * {{ BITS_PER
_VALUE }} / {{ SCRATCH_WRITE_BW_BITS_PER_S }})"
    leak_power: 0
    area: {{ LOCAL_BUFFER_AREA_M2 }}
    actions:
    - {name: read, energy: {{ SCRATCH_READ_ENERGY_J }}, laten
cy: 0}
    - {name: write, energy: {{ SCRATCH_WRITE_ENERGY_J }}, lat
ency: 0}
    tensors: {keep: All, may_keep: All}

  - !Compute
    name: MAC
    total_latency: "compute_actions / ({{ CLOCK_HZ }} * {{ NU
M_MACS }})"
    leak_power: 0
    # Public Arm material gives only a total core-area anchor
for the smallest
    # U55, so we're folding fixed control area and MAC-array
area into one compute block
    area: {{ NPU_CORE_AREA_M2 }}
    actions:
    - {name: compute, energy: {{ MAC_ENERGY_J }}, latency: 0}
```

Creating an Edge YOLO-Workload

My testbench, run_yolo_world.py, gives the total ADC/DAC energy per component, the array energy, and the SRAM/Buffer Energy, which is the cost of moving data from local memory to the compute units.

The head of these models are large 3×3 convolutions on relatively large feature maps (80×80 and 40×40), so I anticipate these to be our most expensive layers. The deeper backbone layers have smaller spatial dimensions (20×20) and this should map very efficiently to CiM arrays because the entire feature map can fit into the local buffers. The bottleneck will be the early layers which proceooss 640×640 or 320×320 images. This causes high SRAM energy usage because the accelerator has to stream the image through the crossbars in many small tiles, leading to high data-movement overhead.

The mapper_script.py handles the tiling and scheduling. I have an Energy-Delay Product (EDP) function, that is the product of energy and latency. The tradeoff is that running things in parallel saves time but will increase peak power or energy due to interconnect overhead. We aim to minimize the EDP to find out where the hardware is most efficient. When run_mapper is called, AccelForge is trying to fit 4D tensors into the physical architecture. Such decisions include how many channels C stay on a crossbar, how many spatial pixels (P, Q) are processed in one cycle, and how many bits of the weight are stored per cell.

Some modeling choices/approximations I made were that I only modeled the inner 3×3 convolutions and disregarded features like the 1×1 splits in the CSPDarknet because they are often negligible for Multiply Accumulates (MACs). The text encoder are modeled as Dense MatMuls. At this scale, the weight-stationary energy (loading the projection weights) will likely dominate over the compute energy because the sequence length is short.

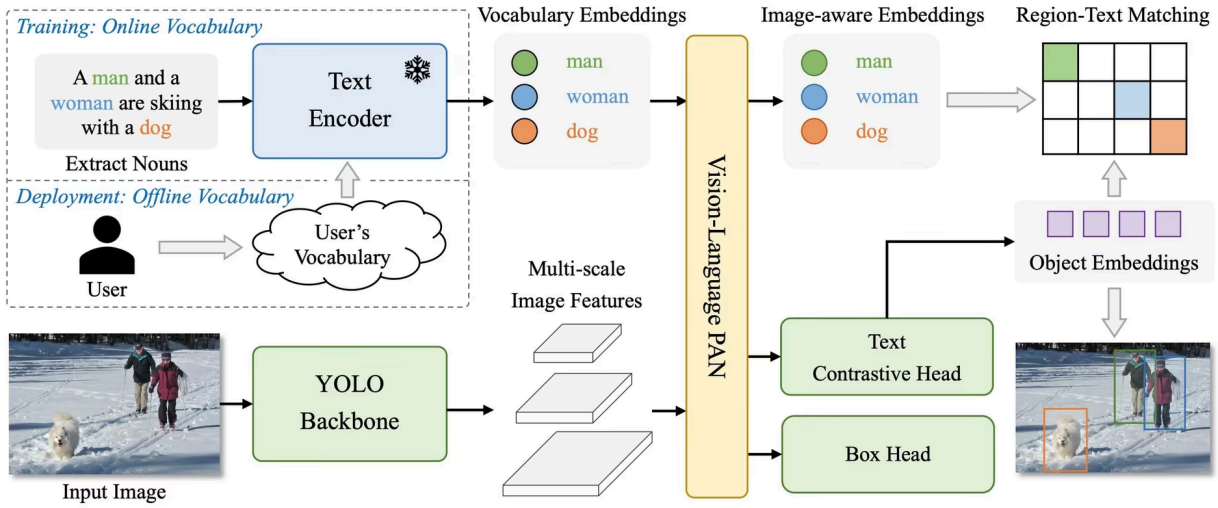


Figure 3. **Overall Architecture of YOLO-World.** Compared to traditional YOLO detectors, YOLO-World as an open-vocabulary detector adopts text as input. The *Text Encoder* first encodes the input text input text embeddings. Then the *Image Encoder* encodes the input image into multi-scale image features and the proposed *RepVL-PAN* exploits the multi-level cross-modality fusion for both image and text features. Finally, YOLO-World predicts the regressed bounding boxes and the object embeddings for matching the categories or nouns that appeared in the input text.

Modular YAML for Workload

```
renames:
  einsums:
  - name: default
    tensor_accesses:
    - name: input
      source: Inputs & Intermediates
      expected_count: 1
    - name: output
      source: Outputs
      expected_count: 1
    - name: weight
      source: ~(input | output)
      expected_count: 1

workload:
  bits_per_value: {All: 8}
  rank_sizes: {N: {{BATCH_SIZE}}}
```

```
  einsums:

  # ── Sanity check ──────────────────────────────────────────
────────────────
  # T0: tiny 3x3 conv  8x8x4 -> 8x8x8
  #   Expected MACs = 8 * 8 * 4 * 8 * 3 * 3 = 18,432
  #   Use run_dnn_layers(..., max_layers=1) to run this alone
first.
  - einsum: "T0[p0, q0, n, m0] = I0[n, c0, H: p0 + r0, W: q0
+ s0] * W0[r0, s0, c0, m0]"
    iteration_space_shape:
    - 0 <= p0 < 8
    - 0 <= q0 < 8
    - 0 <= c0 < 4
    - 0 <= m0 < 8
    - 0 <= r0 < 3
    - 0 <= s0 < 3
    renames: {output: T0, input: I0, weight: W0}

  # ── Backbone: YOLOv8-S stem and stage convolutions ───────
────────────────

  # T1: stem 3x3 stride-2  640x640x3 -> 320x320x32
  #   MACs = 320*320*3*32*3*3 = 88,473,600
  - einsum: "T1[p1, q1, n, m1] = I1[n, c1, H: p1 * 2 + r1, W:
q1 * 2 + s1] * W1[r1, s1, c1, m1]"
    iteration_space_shape:
    - 0 <= p1 < 320
    - 0 <= q1 < 320
    - 0 <= c1 < 3
    - 0 <= m1 < 32
    - 0 <= r1 < 3
    - 0 <= s1 < 3
    renames: {output: T1, input: I1, weight: W1}

  # T2: 3x3 stride-2  320x320x32 -> 160x160x64
```

```
  #   MACs = 160*160*32*64*3*3 = 471,859,200
  - einsum: "T2[p2, q2, n, m2] = I2[n, c2, H: p2 * 2 + r2, W:
q2 * 2 + s2] * W2[r2, s2, c2, m2]"
    iteration_space_shape:
    - 0 <= p2 < 160
    - 0 <= q2 < 160
    - 0 <= c2 < 32
    - 0 <= m2 < 64
    - 0 <= r2 < 3
    - 0 <= s2 < 3
    renames: {output: T2, input: I2, weight: W2}

  # T3: C2f_1 inner bottleneck 3x3 (n=1)  160x160x32 -> 160x1
60x32
  #   MACs = 160*160*32*32*3*3 = 235,929,600
  - einsum: "T3[p3, q3, n, m3] = I3[n, c3, H: p3 + r3, W: q3
+ s3] * W3[r3, s3, c3, m3]"
    iteration_space_shape:
    - 0 <= p3 < 160
    - 0 <= q3 < 160
    - 0 <= c3 < 32
    - 0 <= m3 < 32
    - 0 <= r3 < 3
    - 0 <= s3 < 3
    renames: {output: T3, input: I3, weight: W3}

  # T4: 3x3 stride-2  160x160x64 -> 80x80x128
  #   MACs = 80*80*64*128*3*3 = 471,859,200
  - einsum: "T4[p4, q4, n, m4] = I4[n, c4, H: p4 * 2 + r4, W:
q4 * 2 + s4] * W4[r4, s4, c4, m4]"
    iteration_space_shape:
    - 0 <= p4 < 80
    - 0 <= q4 < 80
    - 0 <= c4 < 64
    - 0 <= m4 < 128
    - 0 <= r4 < 3
```

```
    - 0 <= s4 < 3
    renames: {output: T4, input: I4, weight: W4}

  # T5: C2f_2 inner bottleneck 3x3 (pass a)  80x80x64 -> 80x8
0x64
  #   MACs = 80*80*64*64*3*3 = 235,929,600
  - einsum: "T5[p5, q5, n, m5] = I5[n, c5, H: p5 + r5, W: q5
+ s5] * W5[r5, s5, c5, m5]"
    iteration_space_shape:
    - 0 <= p5 < 80
    - 0 <= q5 < 80
    - 0 <= c5 < 64
    - 0 <= m5 < 64
    - 0 <= r5 < 3
    - 0 <= s5 < 3
    renames: {output: T5, input: I5, weight: W5}

  # T6: C2f_2 inner bottleneck 3x3 (pass b)  80x80x64 -> 80x8
0x64
  #   MACs = 80*80*64*64*3*3 = 235,929,600
  - einsum: "T6[p6, q6, n, m6] = I6[n, c6, H: p6 + r6, W: q6
+ s6] * W6[r6, s6, c6, m6]"
    iteration_space_shape:
    - 0 <= p6 < 80
    - 0 <= q6 < 80
    - 0 <= c6 < 64
    - 0 <= m6 < 64
    - 0 <= r6 < 3
    - 0 <= s6 < 3
    renames: {output: T6, input: I6, weight: W6}

  # T7: 3x3 stride-2  80x80x128 -> 40x40x256
  #   MACs = 40*40*128*256*3*3 = 471,859,200
  - einsum: "T7[p7, q7, n, m7] = I7[n, c7, H: p7 * 2 + r7, W:
q7 * 2 + s7] * W7[r7, s7, c7, m7]"
    iteration_space_shape:
```

```
    - 0 <= p7 < 40
    - 0 <= q7 < 40
    - 0 <= c7 < 128
    - 0 <= m7 < 256
    - 0 <= r7 < 3
    - 0 <= s7 < 3
    renames: {output: T7, input: I7, weight: W7}

  # T8: C2f_3 inner bottleneck 3x3 (pass a)  40x40x128 -> 40x
40x128
  #   MACs = 40*40*128*128*3*3 = 235,929,600
  - einsum: "T8[p8, q8, n, m8] = I8[n, c8, H: p8 + r8, W: q8
+ s8] * W8[r8, s8, c8, m8]"
    iteration_space_shape:
    - 0 <= p8 < 40
    - 0 <= q8 < 40
    - 0 <= c8 < 128
    - 0 <= m8 < 128
    - 0 <= r8 < 3
    - 0 <= s8 < 3
    renames: {output: T8, input: I8, weight: W8}

  # T9: C2f_3 inner bottleneck 3x3 (pass b)  40x40x128 -> 40x
40x128
  #   MACs = 40*40*128*128*3*3 = 235,929,600
  - einsum: "T9[p9, q9, n, m9] = I9[n, c9, H: p9 + r9, W: q9
+ s9] * W9[r9, s9, c9, m9]"
    iteration_space_shape:
    - 0 <= p9 < 40
    - 0 <= q9 < 40
    - 0 <= c9 < 128
    - 0 <= m9 < 128
    - 0 <= r9 < 3
    - 0 <= s9 < 3
    renames: {output: T9, input: I9, weight: W9}
```

```
  # T10: 3x3 stride-2  40x40x256 -> 20x20x512
  #   MACs = 20*20*256*512*3*3 = 471,859,200
  - einsum: "T10[p10, q10, n, m10] = I10[n, c10, H: p10 * 2 +
r10, W: q10 * 2 + s10] * W10[r10, s10, c10, m10]"
    iteration_space_shape:
    - 0 <= p10 < 20
    - 0 <= q10 < 20
    - 0 <= c10 < 256
    - 0 <= m10 < 512
    - 0 <= r10 < 3
    - 0 <= s10 < 3
    renames: {output: T10, input: I10, weight: W10}

  # T11: C2f_4 inner bottleneck 3x3 (n=1)  20x20x256 -> 20x20
x256
  #   MACs = 20*20*256*256*3*3 = 235,929,600
  - einsum: "T11[p11, q11, n, m11] = I11[n, c11, H: p11 + r1
1, W: q11 + s11] * W11[r11, s11, c11, m11]"
    iteration_space_shape:
    - 0 <= p11 < 20
    - 0 <= q11 < 20
    - 0 <= c11 < 256
    - 0 <= m11 < 256
    - 0 <= r11 < 3
    - 0 <= s11 < 3
    renames: {output: T11, input: I11, weight: W11}

  # T12: SPPF 1x1 channel reduction  20x20x512 -> 20x20x256
  #   MACs = 20*20*512*256 = 52,428,800
  - einsum: "T12[p12, q12, n, m12] = I12[n, c12, p12, q12] *
W12[c12, m12]"
    iteration_space_shape:
    - 0 <= p12 < 20
    - 0 <= q12 < 20
    - 0 <= c12 < 512
    - 0 <= m12 < 256
```

```
    renames: {output: T12, input: I12, weight: W12}

  # ── CLIP text encoder (one transformer layer, 77 tokens, d
=512) ────────────

  # T13: Q projection  (77, 512) -> (77, 512)
  #   MACs = 77*512*512 = 20,185,088
  - einsum: "T13[p13, n, m13] = I13[n, c13, p13] * W13[c13, m
13]"
    iteration_space_shape:
    - 0 <= p13 < 77
    - 0 <= c13 < 512
    - 0 <= m13 < 512
    renames: {output: T13, input: I13, weight: W13}

  # T14: K projection  (77, 512) -> (77, 512)
  #   MACs = 77*512*512 = 20,185,088
  - einsum: "T14[p14, n, m14] = I14[n, c14, p14] * W14[c14, m
14]"
    iteration_space_shape:
    - 0 <= p14 < 77
    - 0 <= c14 < 512
    - 0 <= m14 < 512
    renames: {output: T14, input: I14, weight: W14}

  # T15: V projection  (77, 512) -> (77, 512)
  #   MACs = 77*512*512 = 20,185,088
  - einsum: "T15[p15, n, m15] = I15[n, c15, p15] * W15[c15, m
15]"
    iteration_space_shape:
    - 0 <= p15 < 77
    - 0 <= c15 < 512
    - 0 <= m15 < 512
    renames: {output: T15, input: I15, weight: W15}

  # T16: FFN up  (77, 512) -> (77, 2048)
```

```
  #   MACs = 77*512*2048 = 80,740,352
  - einsum: "T16[p16, n, m16] = I16[n, c16, p16] * W16[c16, m
16]"
    iteration_space_shape:
    - 0 <= p16 < 77
    - 0 <= c16 < 512
    - 0 <= m16 < 2048
    renames: {output: T16, input: I16, weight: W16}

  # T17: FFN down  (77, 2048) -> (77, 512)
  #   MACs = 77*2048*512 = 80,740,352
  - einsum: "T17[p17, n, m17] = I17[n, c17, p17] * W17[c17, m
17]"
    iteration_space_shape:
    - 0 <= p17 < 77
    - 0 <= c17 < 2048
    - 0 <= m17 < 512
    renames: {output: T17, input: I17, weight: W17}

  # ── RepVL-PAN cross-modal fusion ──────────────────────────
──────────────────

  # T18: 1x1 image-text projection at P5 scale  20x20x512 ->
20x20x512
  #   MACs = 20*20*512*512 = 104,857,600
  - einsum: "T18[p18, q18, n, m18] = I18[n, c18, p18, q18] *
W18[c18, m18]"
    iteration_space_shape:
    - 0 <= p18 < 20
    - 0 <= q18 < 20
    - 0 <= c18 < 512
    - 0 <= m18 < 512
    renames: {output: T18, input: I18, weight: W18}

  # ── Detection head ────────────────────────────────────────
──────────────────
```

```
  # T19: head 3x3 conv P3  80x80x128 -> 80x80x128
  #   MACs = 80*80*128*128*3*3 = 943,718,400
  - einsum: "T19[p19, q19, n, m19] = I19[n, c19, H: p19 + r1
9, W: q19 + s19] * W19[r19, s19, c19, m19]"
    iteration_space_shape:
    - 0 <= p19 < 80
    - 0 <= q19 < 80
    - 0 <= c19 < 128
    - 0 <= m19 < 128
    - 0 <= r19 < 3
    - 0 <= s19 < 3
    renames: {output: T19, input: I19, weight: W19}

  # T20: head 3x3 conv P4  40x40x256 -> 40x40x256
  #   MACs = 40*40*256*256*3*3 = 943,718,400
  - einsum: "T20[p20, q20, n, m20] = I20[n, c20, H: p20 + r2
0, W: q20 + s20] * W20[r20, s20, c20, m20]"
    iteration_space_shape:
    - 0 <= p20 < 40
    - 0 <= q20 < 40
    - 0 <= c20 < 256
    - 0 <= m20 < 256
    - 0 <= r20 < 3
    - 0 <= s20 < 3
    renames: {output: T20, input: I20, weight: W20}
```